

CELEBAL EXCELLENCE INTERNSHIP PROGRAM

DATA ENGINEERING

WEEK - 2 ASSIGNMENT REPORT - II

E-Commerce Sales Database Analysis using SQL

SUBMITTED BY:-

ANUSHKA SHARMA

(JKLU)

As per the requirements provided in the Week 2 Task document, an E-Commerce Sales Database was created using SQL. The database consists of four tables: customers, products, orders, and order_items. Appropriate Primary Keys, Foreign Keys, constraints, and indexes were implemented according to the given schema.

After creating the tables, the provided sample data was inserted successfully into each table using SQL INSERT statements. The data was then verified to ensure that all records were loaded correctly. Once the database setup was completed, SQL queries were executed to perform data retrieval, filtering, aggregation, joins, and transaction-related operations to generate meaningful business insights.

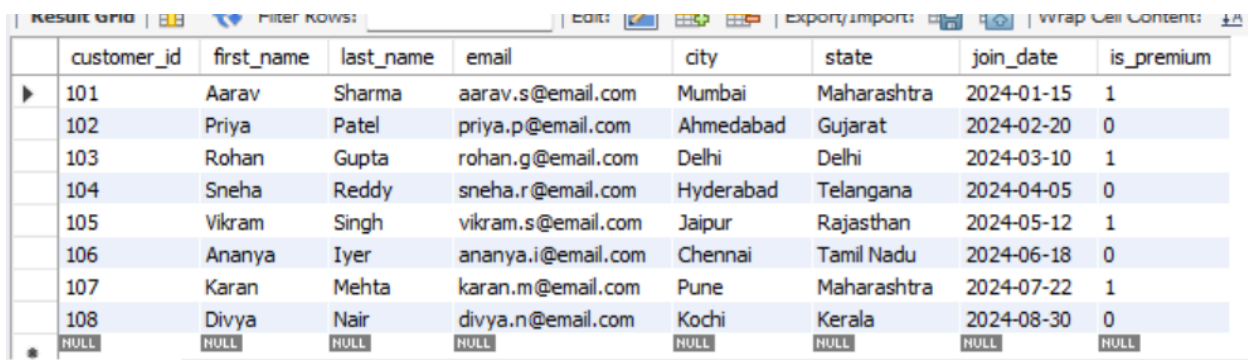
Section A - SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Write a query to display all columns and rows from the customer's table.

Ans.

```
-- Q1
SELECT * FROM customers;
```

Output :-

A screenshot of a SQL query result grid. The grid has a toolbar at the top with icons for filtering, editing, and exporting. Below the toolbar is a table with 9 columns: customer_id, first_name, last_name, email, city, state, join_date, and is_premium. The table contains 9 rows of data, including 8 customer records and one row of NULL values. The first row has a blue arrow icon in the first column.

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Explanation :- Displays complete customer information.

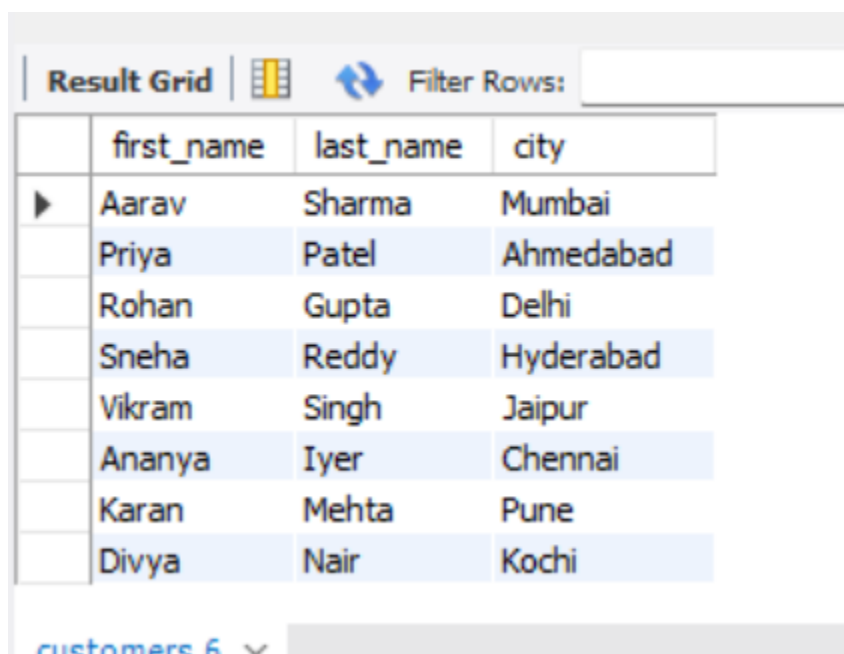
Insight :- The table contains 8 customers from different cities and states across India.

Q2. Retrieve only the first_name, last_name, and city of all customers.

Ans.

```
-- Q2  
SELECT first_name, last_name, city  
FROM customers;
```

Output :-



The screenshot shows a database interface with a 'Result Grid' tab. It displays the results of a SQL query. The grid has four columns: 'first_name', 'last_name', and 'city'. There are nine rows of data. The first row is highlighted with a blue arrow icon in the first column. At the bottom, there is a dropdown menu showing 'customers 6'.

	first_name	last_name	city
▶	Aarav	Sharma	Mumbai
	Priya	Patel	Ahmedabad
	Rohan	Gupta	Delhi
	Sneha	Reddy	Hyderabad
	Vikram	Singh	Jaipur
	Ananya	Iyer	Chennai
	Karan	Mehta	Pune
	Divya	Nair	Kochi

customers 6 ▼

Explanation :- Shows only customer names and their cities.

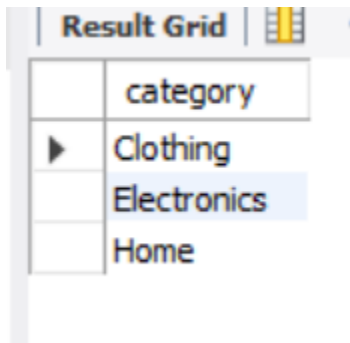
Insight :- Customers are spread across multiple cities, indicating a diverse customer base.

Q3. List all unique categories available in the products table.

Ans.

```
-- Q3  
SELECT DISTINCT category  
FROM products;
```

Output :-



	category
▶	Clothing
	Electronics
	Home

Explanation :- Returns unique product categories.

Insight :- The company sells products in Electronics, Clothing, and Home categories.

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Ans.

Primary key of table customers :- customer_id

Primary key of table products :- product_id

Primary key of table orders :- order_id

Primary key of table order_items :- item_id

A Primary Key uniquely identifies each row in a table. It cannot contain NULL values and duplicate values are not allowed.

Primary Keys help maintain data integrity and prevent duplicate records.

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

Ans.

Constraint :- email VARCHAR(100) UNIQUE NOT NULL

- UNIQUE prevents duplicate emails.
- NOT NULL prevents empty values.

If we try to insert the duplicate email the insertion fails because the email already exists.

Email uniqueness ensures each customer account is distinct.

```
-- Q5
INSERT INTO customers
VALUES (109, 'Test', 'User',
       'aarav.s@email.com',
       'Delhi', 'Delhi',
       '2024-09-01', FALSE);
```

Output :-

```
Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key 'customers.email'
```

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

Ans.

```
-- Q6
INSERT INTO products
VALUES(209, 'Test Product', 'Electronics', 'TestBrand', -50, 100);
```

Output :-

Error Code: 3819. Check constraint 'products_chk_1' is violated.

The query fails due to: CHECK(unit_price > 0)

Negative product prices are not allowed. The CHECK constraint prevents invalid pricing data from entering the database.

Section B - Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

Ans.

```
-- Q7
SELECT *
FROM orders
WHERE status='Delivered';
```

Output :-

Result Grid

Filter Rows:

Edit:

	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1008	103	2024-08-20	Delivered	899.00
	1010	108	2024-08-28	Delivered	1598.00
✱	NULL	NULL	NULL	NULL	NULL

Explanation :- Displays all successfully delivered orders.

Insight :- Most orders in the sample data have been successfully delivered.

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

Ans.

```
-- Q8
SELECT *
FROM products
WHERE category='Electronics'
AND unit_price > 2000;
```

Output :-

Result Grid		Filter Rows:		Edit:		Export/In	
	product_id	product_name	category	brand	unit_price	stock_qty	
▶	203	Smart Watch	Electronics	Noise	2999.00	150	
	205	Bluetooth Speaker	Electronics	JBL	3499.00	200	
*	NULL	NULL	NULL	NULL	NULL	NULL	

Explanation :- Returns high-priced electronic products.

Insight :- Smart Watches and Bluetooth Speakers contribute to higher-value electronics sales.

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

Ans.

```
-- Q9
SELECT *
FROM customers
WHERE state='Maharashtra'
AND join_date BETWEEN '2024-01-01'
AND '2024-12-31';
```

Output :-

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Explanation :- Shows Maharashtra customers who joined during 2024.

Insight :- Maharashtra contributes multiple customers, indicating a strong regional presence.

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

Ans.

```
-- Q10
SELECT *
FROM orders
WHERE order_date BETWEEN '2024-08-10'
AND '2024-08-25'
AND status != 'Cancelled';
```

Output :-

Result Grid					
		Filter Rows:		Edit:	
	order_id	customer_id	order_date	status	total_amount
▶	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1007	106	2024-08-18	Pending	1299.00
	1008	103	2024-08-20	Delivered	899.00
	1009	107	2024-08-25	Shipped	6098.00
•	NULL	NULL	NULL	NULL	NULL

Explanation :- Displays valid orders within the specified date range.

Insight :- The majority of orders during this period were active and processed successfully.

Q11. Explain what the index `idx_orders_date` does. How would it improve the performance of a query that filters orders by `order_date`? Write a sample query that would benefit from this index.

Ans.

`idx_orders_date` is an index on the `order_date` column. It helps the database locate matching dates faster without scanning the entire table.

```
-- Q11
SELECT *
FROM orders
WHERE order_date BETWEEN '2024-08-01'
AND '2024-08-31';
```

Output :-

	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1003	103	2024-08-05	Shipped	7498.00
	1004	101	2024-08-10	Delivered	3499.00
	1005	104	2024-08-12	Cancelled	2999.00
	1006	105	2024-08-15	Delivered	5898.00
	1007	106	2024-08-18	Pending	1299.00
	1008	103	2024-08-20	Delivered	899.00
	1009	107	2024-08-25	Shipped	6098.00
	1010	108	2024-08-28	Delivered	1598.00
•	NULL	NULL	NULL	NULL	NULL
orders 12 ×					

Indexes improve query performance, especially for large datasets with millions of records.

Q12. If you run: `SELECT * FROM customers WHERE YEAR(join_date) = 2024;` would the index on `join_date` be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

Ans.

```
-- Q12
SELECT *
FROM customers
WHERE YEAR(join_date)=2024;
```

Usually the index is not used efficiently because the `YEAR()` function must be applied to every row.

```
-- Better User friendly Query
SELECT *
FROM customers
WHERE join_date >= '2024-01-01'
AND join_date < '2025-01-01';
```

Writing index-friendly queries improves search speed and reduces database workload.

Output :-

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Section C - Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

Ans.

```
-- Q13
SELECT COUNT(*) AS total_orders
FROM orders;
```

Output :-

Result Grid	
	total_orders
▶	10

Explanation :- Counts all orders in the table.

Insight :- The company processed 10 orders in the sample dataset.

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

Ans.

```
-- Q14
SELECT SUM(total_amount) AS total_revenue
FROM orders
WHERE status='Delivered';
```

Output :-

Result Grid	
	total_revenue
▶	17191.00

Explanation :- Calculates total revenue generated from delivered orders only.

Insight :- Delivered orders generate the actual realized revenue for the business.

Q15. Calculate the average unit_price of products in each category.

Ans.

```
-- Q15
SELECT category,
AVG(unit_price) AS avg_price
FROM products
GROUP BY category;
```

Output :-

	category	avg_price
►	Clothing	2699.000000
	Electronics	2224.000000
	Home	949.000000

Explanation :- Calculates average price for each category.

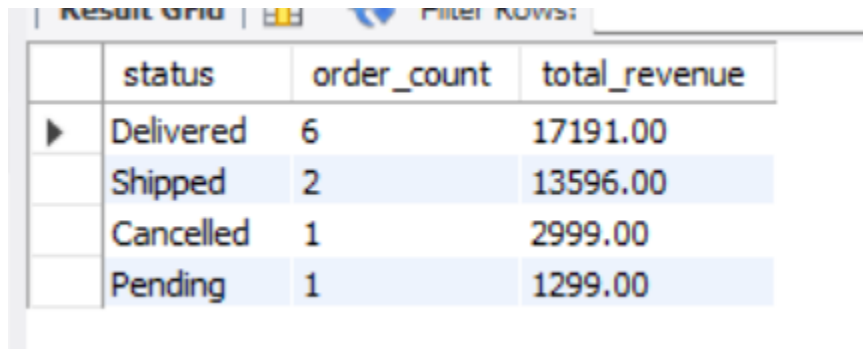
Insight :- Electronics products have a higher average price compared to Home products.

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

Ans.

```
-- Q16
SELECT status,
COUNT(*) AS order_count,
SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
```

Output :-



The screenshot shows a database result grid with the following data:

	status	order_count	total_revenue
▶	Delivered	6	17191.00
	Shipped	2	13596.00
	Cancelled	1	2999.00
	Pending	1	1299.00

Explanation :- Shows order volume and revenue generated by each order status.

Insight :- Delivered orders contribute the highest revenue, while cancelled orders add no business value.

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

Ans.

```
-- Q17
SELECT category,
MAX(unit_price) AS most_expensive,
MIN(unit_price) AS cheapest
FROM products
GROUP BY category;
```

Output :-

	category	most_expensive	cheapest
►	Clothing	4599.00	799.00
	Electronics	3499.00	899.00
	Home	1299.00	599.00

Explanation :- Finds price extremes within each category.

Insight :- There is a significant price difference between premium and budget products within categories.

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

Ans.

```
-- Q18
SELECT category,
AVG(unit_price) AS avg_price
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;
```

Output :-

	category	avg_price
►	Clothing	2699.000000
	Electronics	2224.000000

Explanation :- Filters categories based on average product price.

Insight :- Electronics is the premium category with the highest average selling price.

Section D - Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

Ans.

```
-- Q19
SELECT o.order_id,
       o.order_date,
       c.first_name,
       c.last_name,
       o.total_amount
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id;
```

Output :-

Result Grid					
Filter Rows:					
Export:  Write					
	order_id	order_date	first_name	last_name	total_amount
▶	1001	2024-08-01	Aarav	Sharma	4498.00
	1004	2024-08-10	Aarav	Sharma	3499.00
	1002	2024-08-03	Priya	Patel	799.00
	1003	2024-08-05	Rohan	Gupta	7498.00
	1008	2024-08-20	Rohan	Gupta	899.00
	1005	2024-08-12	Sneha	Reddy	2999.00
	1006	2024-08-15	Vikram	Singh	5898.00
	1007	2024-08-18	Ananya	Iyer	1299.00
	1009	2024-08-25	Karan	Mehta	6098.00
	1010	2024-08-28	Divya	Nair	1598.00

Explanation :- Combines the orders and customers tables to display order details along with customer names.

Insight :- This helps identify which customer placed each order and how much they spent.

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

Ans.

```
-- Q20
SELECT c.customer_id,
       c.first_name,
       c.last_name,
       o.order_id,
       o.order_date
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

Output :-

Result Grid					
Filter Rows:					
Export:					
	customer_id	first_name	last_name	order_id	order_date
▶	101	Aarav	Sharma	1001	2024-08-01
	101	Aarav	Sharma	1004	2024-08-10
	102	Priya	Patel	1002	2024-08-03
	103	Rohan	Gupta	1003	2024-08-05
	103	Rohan	Gupta	1008	2024-08-20
	104	Sneha	Reddy	1005	2024-08-12
	105	Vikram	Singh	1006	2024-08-15
	106	Ananya	Iyer	1007	2024-08-18
	107	Karan	Mehta	1009	2024-08-25
	108	Divya	Nair	1010	2024-08-28

Explanation :- Returns all customers. If a customer has no order, order-related columns show NULL.

Insight :- Useful for identifying customers who have registered but never placed an order.

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

Ans.

```
-- Q21
SELECT o.order_id,
       p.product_name,
       oi.quantity,
       oi.unit_price,
       oi.discount_pct
FROM orders o
JOIN order_items oi
ON o.order_id = oi.order_id
JOIN products p
ON oi.product_id = p.product_id;
```

Output :-

Result Grid					
		Filter Rows:		Export:	Wrap Cell Content:
	order_id	product_name	quantity	unit_price	discount_pct
▶	1001	Wireless Earbuds	2	1499.00	0.00
	1006	Wireless Earbuds	1	1499.00	10.00
	1002	Cotton T-Shirt	1	799.00	0.00
	1003	Smart Watch	1	2999.00	0.00
	1005	Smart Watch	1	2999.00	0.00
	1003	Running Shoes	1	4599.00	5.00
	1006	Running Shoes	1	4599.00	5.00
	1004	Bluetooth Speaker	1	3499.00	0.00
	1009	Bluetooth Speaker	1	3499.00	0.00
	1007	Bedsheet Set	1	1299.00	0.00
	1010	Bedsheet Set	1	1299.00	0.00
	1001	Laptop Stand	1	899.00	10.00
	1008	Laptop Stand	1	899.00	0.00
	1009	Cushion Covers (...)	2	599.00	15.00
	1010	Cushion Covers (...)	1	599.00	0.00

Explanation :- Combines orders, order_items, and products tables to show detailed information about each purchased item.

Insight :- This query helps analyze product-level sales and discounts.

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

Ans.

LEFT JOIN :- Returns all records from the left table and matching records from the right table.

Example:

```
-- Q22
-- LEFT JOIN
SELECT *
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

Output :-

customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id	orde
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101	2024
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1004	101	2024
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1002	102	2024
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1003	103	2024
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1008	103	2024
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104	2024
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1	1006	105	2024
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0	1007	106	2024
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1	1009	107	2024
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0	1010	108	2024

All customers will appear even if they have not placed any order.

RIGHT JOIN :- Returns all records from the right table and matching records from the left table.

Example:

```
-- RIGHT JOIN
SELECT *
FROM customers c
RIGHT JOIN orders o
ON c.customer_id = o.customer_id;
```

Output :-

Result Grid											
Filter Rows:											
Export:											
Wrap Cell Content: ☐											
	customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id	on
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101	20%
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1002	102	20%
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1003	103	20%
	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1004	101	20%
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104	20%
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1	1006	105	20%
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0	1007	106	20%
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1008	103	20%
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1	1009	107	20%
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0	1010	108	20%

All orders will appear even if customer information is missing.

FULL OUTER JOIN :- Returns all matching records plus all non-matching records from both tables. Rows without a match on either side will contain NULL values.

When would you use it? A FULL OUTER JOIN is useful when you want to see every customer and every order in a single result, including customers with no orders and orders without matching customer records.

Insight :- LEFT JOIN is commonly used in business analysis to identify customers who have not made purchases, while FULL OUTER JOIN helps find missing or unmatched data across tables.

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

Ans.

Foreign Key	References
orders.customer_id	customers.customer_id
order_items.order_id	orders.order_id

order_items.product_id	Products.product_id
------------------------	---------------------

```
-- Q23
INSERT INTO orders
VALUES(1011,999,'2024-09-01','Pending',1000);
```

Output :-

Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('shop.

The query fails because customer_id 999 does not exist in the customers table. The Foreign Key constraint prevents invalid references.

Foreign Keys maintain data consistency between related tables.

Section E - Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

Ans.

```
-- Q24
SELECT product_name, unit_price,
CASE
    WHEN unit_price < 1000 THEN 'Budget'
    WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
    ELSE 'Premium'
END AS price_tier
FROM products;
```

Output :-

Result Grid			
Filter Rows:			
	product_name	unit_price	price_tier
▶	Wireless Earbuds	1499.00	Mid-Range
	Cotton T-Shirt	799.00	Budget
	Smart Watch	2999.00	Mid-Range
	Running Shoes	4599.00	Premium
	Bluetooth Speaker	3499.00	Premium
	Bedsheet Set	1299.00	Mid-Range
	Laptop Stand	899.00	Budget
	Cushion Covers (Set)	599.00	Budget

Explanation :- Classifies products into Budget, Mid-Range, and Premium categories based on price.

Insight :- Most electronics products fall into the Mid-Range or Premium category.

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

Ans.

```
-- Q25
SELECT
> SUM(
>     CASE
>         WHEN status='Delivered' THEN 1
>         ELSE 0
>     END
- ) AS delivered_orders,

> SUM(
>     CASE
>         WHEN status<>'Delivered' THEN 1
>         ELSE 0
>     END
- ) AS not_delivered_orders
FROM orders;
```

Output :-

	delivered_orders	not_delivered_orders
▶	6	4

Explanation :- Counts delivered orders and all remaining orders in a single row.

Insight :- Delivered orders indicate successful sales, while other statuses help track operational progress.

Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

Ans.

A - Atomicity :- A transaction is completed fully or not completed at all.

Example: During a bank transfer, money should be deducted and deposited together. If one step fails, the entire transaction is cancelled.

C - Consistency :- The database should remain accurate and valid before and after a transaction.

Example: Total money in the banking system remains correct after a transfer.

I - Isolation :- Multiple transactions should not interfere with each other.

Example: Two customers withdrawing money simultaneously should not affect each other's transactions.

D - Durability :- Once a transaction is committed, the data is permanently saved.

Example: Even if the system crashes after a transfer, the transaction remains recorded.

Insight :- ACID properties ensure database reliability, accuracy, and protection against data loss.

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order
3. Update the stock_qty of the purchased products

4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.

Write the complete BEGIN...COMMIT/ROLLBACK block.

Ans.

```
-- Q27
START TRANSACTION;

INSERT INTO orders VALUES(1011,102,CURDATE(),'Pending',1598.00);

INSERT INTO order_items VALUES(5016,1011,206,1,1299.00,0);

INSERT INTO order_items VALUES(5017,1011,208,1,599.00,0);

UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 206;

UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 208;

COMMIT;
```

Output :-

✓	130	15:28:21	START TRANSACTION	0 row(s) affected	0.000 sec
✓	131	15:28:21	INSERT INTO orders VALUES (1011,102,CURDATE(), 'Pending',1598.00)	1 row(s) affected	0.015 sec
✓	132	15:28:21	INSERT INTO order_items VALUES (5016,1011,206,1,1299.00,0)	1 row(s) affected	0.000 sec
✓	133	15:28:21	INSERT INTO order_items VALUES (5017,1011,208,1,599.00,0)	1 row(s) affected	0.000 sec
✓	134	15:28:21	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
✓	135	15:28:21	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
✓	136	15:28:21	COMMIT	0 row(s) affected	0.016 sec

Explanation :- The transaction inserts a new order, adds its order items, updates inventory, and saves all changes together. If any step fails, all changes are reversed.

Insight :- Transactions prevent partial updates and ensure that orders and inventory remain synchronized.

